RISKORA

AI-assisted micro-lending risk assessment and decision support

Integrated project technical review

Report date: 2026-10-10 | Model: riskora-ml-v1.0

Scope and evidence

This report documents the integrated reviewer workflow, API, database, model, verification and remaining work. The model uses synthetic data. Funding and receipts represent application records, not bank transfers. Human review remains required.

Prepared from maintained repository documentation and the saved model metrics. No student, institution or guide identity has been invented.

Student name(s): _____

Registration number(s): _____

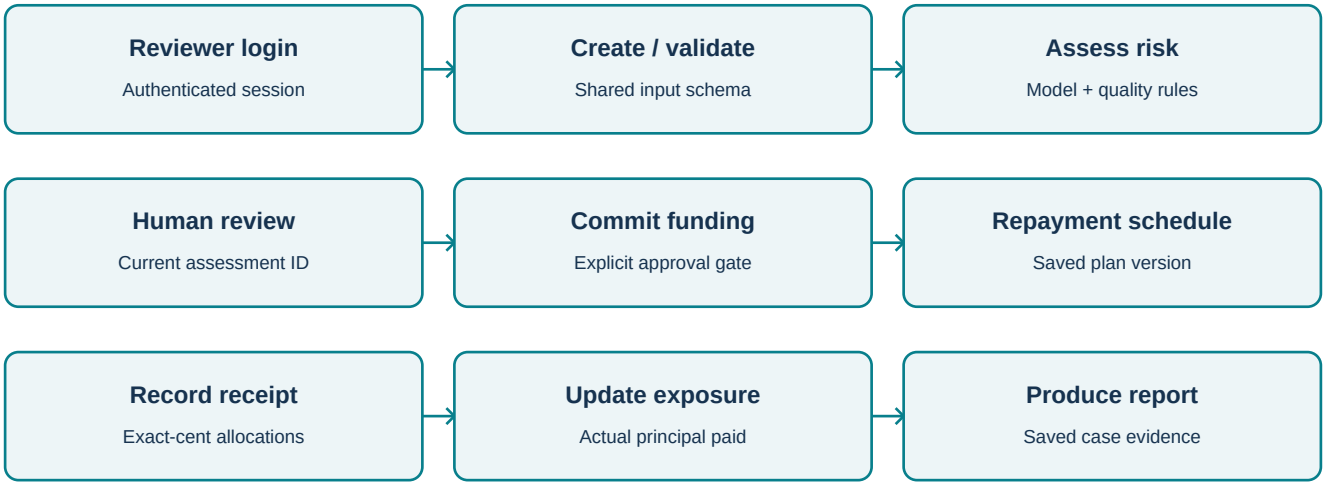
Institution / course: _____

Guide: _____

Reading guide

1. Project overview and workflow - source: README.md
2. System architecture - source: ARCHITECTURE.md
3. Borrower data contract - source: DATA_DICTIONARY.md
4. Database and accounting records - source: DATABASE_SCHEMA.md
5. API contracts - source: API_DOCUMENTATION.md
6. Machine learning pipeline - source: ML_PIPELINE.md
7. Model evaluation and limitations - source: MODEL_CARD.md
8. OraAgent integration - source: INTEGRATION.md
9. Demonstration procedure - source: DEMO_GUIDE.md
10. Verification and evidence - source: TESTING.md
11. Remaining work and priorities - source: PROJECT_ANALYSIS.md

Workflow at a glance



Read each row from left to right, then continue on the next row. All financial actions are simulated.

The following chapters are generated from the source guides. Detailed Mermaid component, state and relationship diagrams remain editable in ARCHITECTURE.md and DATABASE_SCHEMA.md. This PDF is a release snapshot; regenerate it when documentation or model metadata changes.

1. Project overview and workflow

Source: README.md

AI-assisted micro-lending risk assessment and decision support. RISKORA is a local reviewer workstation built with Python, Tornado, SQLite and a browser frontend. It combines a synthetic-data credit-risk model with explicit human review, simulated funding commitments, repayment schedules and a receipt ledger.

The October 2026 OraAgent integration adds authenticated reviewer accounts, borrower create/edit forms, evidence summaries, recorded review decisions and partial-payment handling to the existing application. It uses one backend and one database. See INTEGRATION.md for the source archive and adaptation choices.

Scope: this is an academic application and workflow prototype. Predictions are trained on synthetic data. Funding positions are simulated, and recording a receipt does not move money. ORA is a deterministic evidence summary, not an autonomous agent or an external verification service.

Configuration

`.env.example` is a reference, **not an automatically loaded configuration file**. Set variables in the shell before starting Python. Relative paths resolve from the repository root.

Variable	Default	Purpose
`RISKORA_HOST`	`127.0.0.1`	HTTP bind address; local-only by default
`RISKORA_PORT`	`8000`	Preferred port; positional CLI port takes precedence
`RISKORA_DB_PATH`	`data/runtime/riskora.db`	Mutable SQLite database
`RISKORA_DATA_PATH`	`data/loan_default_full.csv`	Synthetic dataset used by the explorer
`RISKORA_ARTIFACTS_DIR`	`ml/artifacts`	Matching model, preprocessor and metrics directory

Example: use a separate demonstration database without touching normal reviewer data.

```
$env:RISKORA_DB_PATH = 'data/runtime/faculty-demo.db'
.\.venv\Scripts\python.exe run.py 8010
Remove-Item Env:RISKORA_DB_PATH
```

On POSIX shells: `RISKORA_DB_PATH=data/runtime/faculty-demo.db .venv/bin/python run.py 8010`.

The default first startup copies the historical tracked `data/riskora.db` into `data/runtime/riskora.db` if the runtime file is absent. Custom database paths do not import that historical database. An empty database receives six demonstration loans and a historical reviewer label, but no login credentials. Runtime accounts, sessions and new transactions remain in the ignored runtime directory.

Reviewer workflow

1. **Create a loan.** Enter the borrower name and model inputs. Income is annual INR; loan term and employment tenure are months; DTI is a ratio. The frontend loads allowed values from `/api/schema` and the backend validates them again.

2. **Assess the saved case.** The service runs input-quality checks, anomaly indicators and the calibrated model. Missing optional intake fields remain visibly missing. An assessment can exist while the case still needs verification.

3. **Review the evidence.** ORA shows the saved model result, input snapshot reference and data-quality concerns. Record APPROVED, REJECTED or NEEDS_VERIFICATION with a note. Approval requires complete inputs and the latest assessment.

4. **Commit funding.** A preview does not save a commitment. The saved action requires approval and the exact saved principal. Splitting participation changes concentration, not borrower default probability.

5. **Generate a schedule.** Use the approved principal, rate and term. The server restores lender positions from its database. A replacement schedule is permitted before any receipts; afterward it is blocked.

6. **Record a receipt.** Supply the installment, positive amount and a unique payment reference. Partial receipts are supported. Retry an uncertain submission with the same reference, not a new reference.

7. **Resume and report.** Reloading a case restores saved funding, schedule, receipts and outstanding principal. Review exposure and open the decision report. Browser printing can save a PDF of that case.

The implemented stored stages are `REQUEST_RECEIVED`, `NEEDS_VERIFICATION`, `RISK_ASSESSED`, `APPROVED`, `REJECTED`, `FUNDED`, `REPAYMENT_CONFIGURED`, `REPAYING` and `CLOSED`. Separate persisted Draft and Submitted stages are future work. Editing an unfunded case requires reassessment; reanalysis does not reset a funded or repaying case to an intake stage. See ARCHITECTURE.md for transition guards.

Risk and accounting semantics

Display	Source and meaning
Probability of default	Calibrated HistGradientBoosting model on the normalized input
Risk band	LOW below 8% PD, MEDIUM from 8% to below 20%, HIGH at 20% or above
Display score	Piecewise transformation of PD, bounded to 5-99; not a bureau score
Driver/protective points	Fixed policy-rule indicators, not SHAP values or model contributions
Global coefficient chart	Logistic-regression baseline coefficients, not the deployed tree model
Dataset explorer estimate	Separate heuristic estimate over synthetic records; not saved-case inference
ORA brief	Deterministic summary of available assessment and borrower inputs
Concentration	Distribution of a loan across simulated lender positions
Outstanding principal	Original principal less principal allocated from actual recorded receipts

Money is displayed with two decimal places. Payment allocation uses integer cents internally so each receipt exactly reconciles across lender principal and interest buckets. Interest paid does not reduce outstanding principal. The SQLite schema retains REAL amounts and JSON schedule structures; it is not a general-purpose double-entry ledger.

Historical receipts lacking a schedule link cannot be safely allocated by guessing an installment number. Those cases show a reconciliation warning, return unknown outstanding principal and block further receipts. The application does not yet include a reconciliation editor, refund flow, charge-off flow or bank settlement integration.

2. System architecture

Source: ARCHITECTURE.md

This document describes the integrated application as implemented in October 2026. Module names and stored states refer to the running Tornado system. The OraAgent ZIP contributed workflow ideas and adapted components; its separate FastAPI application is not a second service in this architecture.

System context

One Python process serves both the frontend and API. There is no required cloud service, LLM API, message broker, bank API or frontend build service. SQLite holds workflow state. The synthetic CSV supports training and exploration; it is not the loan transaction database.

Component responsibilities

Component	Responsibility	Important boundary
`run.py`	Validate assets and start the server	Never silently retrains
`backend/config.py`	Resolve environment settings	Paths independent of launch directory
`backend/server.py`	HTTP orchestration, risk, funding, repayment and analytics	Checks workflow state before writes
`backend/integration.py`	Setup, accounts, profile edits, human review and audit reads	Review tied to current assessment
`backend/schema.py`	Canonical names, aliases, conversion, validation and inference defaults	Missing storage values are not invented
`backend/security.py`	Password hashes and session creation/lookup	Raw session token stored only in cookie
`backend/database.py`	Connection lifecycle, schema and demo seed	Foreign keys enabled on each connection
`backend/services/data_quality.py`	Completeness and consistency checks	Does not verify source documents
`backend/services/anomaly_detector.py`	Rule/statistical anomaly indicators	Flag for review, not proof of fraud
`backend/services/funding_service.py`	Simulated lender positions and concentration	Cannot change borrower PD
`backend/services/repayment_service.py`	Schedule and planned lender allocations	Approved terms remain authoritative
`backend/services/payment_service.py`	Actual partial receipt allocation and ledger	Integer-cent reconciliation
`backend/services/ora_service.py`	Evidence brief from saved fields and assessment	No fabricated external evidence
`ml/infer.py`	Preprocess, predict PD, map band and policy points	Only trusted serialized artifacts
`app.js`	Main application views, selection and reports	Restores server-owned case state
`integration.js`	Authentication and new workflow dialogs	Calls the same validated API

Consult the directory for exact service filenames if refactoring. The conceptual boundaries matter more than creating a service per diagram box.

Request and data flow

The browser may validate early to help the reviewer, but the server validates again. Saved analysis loads database inputs. Conflicting analysis overrides are rejected; a real correction must use the edit endpoint and a hypothetical correction must use simulation. Alias normalization is shared with inference, so different field spellings do not create separate risk models.

Stored lifecycle and guards

This diagram shows the normal path; handlers also permit reanalysis of early-stage cases. Editing any unfunded case returns it to `REQUEST_RECEIVED`. Reanalysis of an approved or rejected case requires a new review before funding. Reanalysis after funding may add an assessment but preserves the financial stage. There is no saved Draft/Submitted split yet.

Action	Guard	Effect
Edit profile	No funding commitment exists	Save full replacement, reset assessment eligibility
Approve	Current stage assessed/needs verification; latest assessment ID; quality ready	Insert decision with reviewer identity
Preview funding	No loan ID	Return structure without persistence
Commit funding	Approved, exact principal, no previous commitment	Save positions and set FUNDED
Generate schedule	Funded/configured; exact saved terms; no receipts	Append schedule version and set configured
Record receipt	Current schedule, valid amount/reference, no ambiguous legacy receipts	Save receipt, allocations and stage atomically
Close	Every installment fully paid	Set CLOSED

Changing a selector cannot authorize a financial write. Funding and payment routes use server-side values and explicit calls. No action in this prototype transfers real funds.

Financial persistence

A schedule is a plan. A payment is an actual recorded event. Outstanding principal is derived from actual principal allocations; it is not taken from the balance of the last scheduled row. Interest and principal are tracked separately.

The receipt operation obtains a SQLite write transaction with `BEGIN IMMEDIATE`, checks idempotency and remaining installment buckets, allocates integer cents, inserts the receipt, updates installment state and writes the audit event. Matching retries return the existing result. The unique reference index provides an additional database guard.

Schedule history remains in `repayment_schedules`. Its row ID acts as the version. The latest version is active, and a client must send its ID for payment recording. Schedules cannot be regenerated after receipts. Historical unlinked receipts are preserved and flagged for reconciliation rather than assigned to an arbitrary schedule.

Security boundaries

Public pages include the login shell and allowlisted assets. Business API data requires a session. Public API exceptions are health, login and first-use setup. ADMIN and REVIEWER can perform case actions; VIEWER can read and log out; account creation is administrator-only.

Every POST requires the custom request header. A supplied Origin must match the serving scheme and host. Cookies are HttpOnly and SameSite=Strict; Secure is set when Tornado sees HTTPS. Passwords use salted PBKDF2, while session tokens are random and stored as hashes. These controls do not provide TLS by themselves. A future proxy must be configured and tested so HTTPS, source IP and origin handling are correct.

Static serving is an allowlist, not a repository directory server. Source code, database files, environment files, CSV data and model artifacts are not served as static downloads. Model artifacts are trusted build inputs: joblib loading is unsuitable for untrusted uploaded files.

Browser state and recovery

The server owns profiles, assessments, commitments, schedules and receipts. Browser state stores the active selection and temporary previews. Active-case sequence guards prevent late requests from painting a previous case over the current one. Navigation reloads relevant reports, and switching cases clears prior-case content.

Authentication dialogs live in `integration.js`; the application shell is hidden and inert when signed out. Intake forms load schema values from the backend. Payment dialogs collect amount and reference. Mobile navigation exposes state through accessibility attributes and supports the smaller viewport. The browser is not the security boundary: hiding a button must never substitute for backend authorization.

Deployment and scaling limits

This is a single-process local architecture. Inference and SQLite operations run synchronously in HTTP handlers. PBKDF2 also consumes CPU. A threaded database, background task queue, distributed session service and connection pool are not implemented. Rate limiting is process-local and resets on restart. Load testing is required before claiming concurrent production capacity.

The health endpoint is public service status, not a deep readiness probe of database writes, backup health or external dependencies. Structured operational logs, managed migrations, tested restoration automation, account recovery and a supported HTTPS deployment package remain planned work. See PROJECT_ANALYSIS.md for the ordered plan.

3. Borrower data contract

Source: DATA_DICTIONARY.md

`backend/schema.py` is the source of truth for borrower fields across create/edit, saved analysis, inference and simulation. `GET /api/schema` exposes numeric constraints, categories and required intake fields to the frontend. The model has 16 raw inputs: nine numeric and seven categorical.

Required intake and complete assessment

Eight numeric fields are required to save a new application. Age and seven categorical fields may be absent at intake, but approval requires all 16 values plus passing consistency rules. A borrower name is also required for creation/editing: a trimmed nonempty string of 1-200 characters. Name and server-generated IDs are not model predictors.

Blank strings and JSON null are treated as missing. Numeric booleans, nonfinite values, invalid ranges, fractional integers and unknown categories are rejected. Categories are matched case-insensitively and stored in canonical form. Yes/No fields also accept true/false and 1/0 aliases. Unrecognized extra keys are ignored by profile normalization; clients should still send only documented fields.

Numeric fields

Field	Unit	Inclusive range	Integer	Required at intake	Inference fallback
`Age`	years	18 to 95	Yes	No	35
`Income`	annual INR	1 to 5000000	No	Yes	450000
`LoanAmount`	INR, at most 2 decimals	0.01 to 5000000	No	Yes	120000
`CreditScore`	input credit score	300 to 850	Yes	Yes	650
`MonthsEmployed`	months	0 to 948	Yes	Yes	36
`NumCreditLines`	count	0 to 25	Yes	Yes	4
`InterestRate`	annual percent, 9.5 means 9.5%	0 to 100	No	Yes	12.5
`LoanTerm`	months	1 to 120	Yes	Yes	36
`DTIRatio`	ratio, 0.16 means 16%	0 to 1.5	No	Yes	0.32

Categorical fields

Field	Allowed values	Inference fallback
`Education`	High School, Bachelor's, Master's, PhD	Bachelor's
`EmploymentType`	Salaried, Self-employed, Contract, Unemployed	Salaried
`MaritalStatus`	Single, Married, Divorced	Married
`LoanPurpose`	Business, Education, Home, Auto, Other	Business
`HasMortgage`	Yes, No	No
`HasDependents`	Yes, No	No
`HasCoSigner`	Yes, No	No

Accepted field names

Use canonical names in new clients. Existing aliases remain supported. Supplying two aliases with different normalized values is an error, not last-write-wins behavior. Scenario overrides are normalized separately before they replace baseline fields.

Canonical field	Other accepted names
`Age`	`age`
`Income`	`income`, `annualIncome`
`LoanAmount`	`loan_amount`, `loanAmount`, `amount`
`CreditScore`	`credit_score`, `creditScore`, `credit`
`MonthsEmployed`	`months_employed`, `monthsEmployed`, `empMonths`
`NumCreditLines`	`num_credit_lines`, `numCreditLines`, `creditLines`
`InterestRate`	`interest_rate`, `interestRate`, `rate`
`LoanTerm`	`loan_term`, `loanTerm`, `term`
`DTIRatio`	`dti_ratio`, `dtiRatio`, `dti`
`Education`	`education`
`EmploymentType`	`employment_type`, `employmentType`, `employment`
`MaritalStatus`	`marital_status`, `maritalStatus`
`HasMortgage`	`has_mortgage`, `hasMortgage`
`HasDependents`	`has_dependents`, `hasDependents`
`LoanPurpose`	`loan_purpose`, `loanPurpose`, `purpose`
`HasCoSigner`	`has_cosigner`, `hasCoSigner`, `hasCosigner`

Missing values and quality rules

Fallbacks exist only in the inference adapter; they are not a claim that a missing input is known. Database storage, input snapshots and quality results retain the missingness. A model result produced using fallbacks must be read alongside its missing-data warnings.

The quality score combines required-field completeness (65 points), optional-field completeness (15 points) and a base 20 points, subtracting 5 per warning and 12 per contradiction, then rounding and clamping to 10-100. Tiers are excellent at 88+, acceptable at 70+, degraded at 50+, otherwise critical. Approval uses `verification_ready`, not only this numeric score.

Contradictions include employment tenure longer than `(Age - 16) * 12`, an unemployed category with positive employment months, and zero credit lines with DTI above 0.35. Warnings include income below INR 50,000 and a loan above 2.5 times annual income without a co-signer. Verification readiness requires no missing inputs, warnings or contradictions. This is internal input consistency, not bank or identity verification.

Dataset-only fields and derived features

Field	Meaning	Used as predictor?
`LoanID`	Synthetic row identifier	No
`Default`	Synthetic binary label, 1=default	Target only
`LoanToIncome`	LoanAmount / annual Income, denominator protected at 1	Derived predictor
`MonthlyBurdenRatio`	Principal per month / monthly income, denominator protected at 1	Derived predictor

The dataset contains 12,000 generated rows. Its values are not real customer observations. Application borrower/loan IDs (`BRW-...`, `LR-...`) identify SQLite records independently of dataset LoanID values.

Assessment outputs

Field	Unit/meaning
`probability_of_default`	Fraction from 0 to 1, returned rounded to four decimal places
`probability_pct`	Percentage, returned rounded to two decimal places
`risk_score`	Display transformation bounded to 5-99, not a bureau score
`risk_level`	LOW, MEDIUM or HIGH from model thresholds
`data_quality_score`	Input-quality score, not model accuracy
`review_level`	Suggested review intensity; does not authorize funding
`impact_pts`	Fixed policy-rule points; not additive model contributions
`model_version`	Artifact metadata version attached to saved assessment
`input_snapshot_json`	Normalized saved inputs, potentially including missing values

Band calculations use the underlying probability before display rounding. A printed probability near a boundary may round to 8.00% or 20.00%; the stored band remains the classification returned by the model engine.

Funding, schedule and receipt terms

`amount` and `principal` are INR. Schedule `rate` is annual percent and `term` is months. `lenderCount` is an integer from 2 to 6 for fractional participation; single funding has one position. Concentration describes lender participation and is independent of borrower default probability.

`scheduleId` is the active repayment_schedules row ID. `installmentNum` is a positive whole installment number. Payment `amount` accepts a number or decimal string with at most two places. `idempotencyKey` is a client reference of 8-100 characters and must be reused for retries of the same receipt.

`received` includes principal and interest. `principal_paid` includes only principal allocations. `outstanding_principal` is original principal less actual principal receipts; it can be null when legacy records need reconciliation. Null does not mean zero. `PARTIAL` and `PAID` describe installment receipt status, while `REPAYING` and `CLOSED` describe loan workflow status.

Source ownership

The reviewer supplies borrower fields. The model supplies PD and band. Policy code supplies rule indicators. Funding and repayment services supply simulated plans. Recorded receipts supply actual application-ledger balances. ORA labels those sources and computes an input digest; it does not verify income, identity, account ownership or a payment against an external institution.

4. Database and accounting records

Source: DATABASE_SCHEMA.md

SQLite is the application store. The default mutable file is `data/runtime/riskora.db`; it is ignored by Git. `data/riskora.db` is a historical demonstration database, copied only on the first default startup. Tests create temporary databases and do not modify either application file.

Relationships

The diagram shows declared foreign keys. `payments.schedule_id`, `risk_assessments.reviewer_id` and audit entity/reviewer references are application-managed references, not declared foreign keys. The legacy `users` table is separate from login accounts.

Table definitions

The tables below were generated from `PRAGMA table_info` and `PRAGMA foreign_key_list` on a fresh isolated database initialized by the current code. PK marks the declared primary key. Required reflects the column's explicit NOT NULL flag; SQLite primary-key behavior must also be considered. No claim is made that domain ranges are database CHECK constraints: those are validated by the API.

`users`

Historical reviewer labels. These rows cannot sign in.

Column	SQLite type	PK	Required	Default	Foreign key
id	TEXT	Yes	No	None	None
username	TEXT	No	Yes	None	None
full_name	TEXT	No	Yes	None	None
role	TEXT	No	Yes	'RISK_UNDERWRITER'	None
created_at	TIMESTAMP	No	No	CURRENT_TIMESTAMP	None

`borrowers`

Borrower identity and demographic/employment inputs. Missing optional intake values remain NULL.

Column	SQLite type	PK	Required	Default	Foreign key
id	TEXT	Yes	No	None	None
name	TEXT	No	Yes	None	None
age	INTEGER	No	No	None	None
income	REAL	No	Yes	None	None
education	TEXT	No	No	None	None
employment_type	TEXT	No	No	None	None
months_employed	INTEGER	No	No	None	None
marital_status	TEXT	No	No	None	None

Column	SQLite type	PK	Required	Default	Foreign key
has_dependents	TEXT	No	No	'No'	None
has_mortgage	TEXT	No	No	'No'	None
has_cosigner	TEXT	No	No	'No'	None
created_at	TIMESTAMP	No	No	CURRENT_TIMESTAMP	None

`loans`

Requested loan terms and current workflow stage. New IDs use an LR- prefix and UUID suffix.

Column	SQLite type	PK	Required	Default	Foreign key
id	TEXT	Yes	No	None	None
borrower_id	TEXT	No	Yes	None	borrowers.id (delete CASCADE)
loan_amount	REAL	No	Yes	None	None
credit_score	INTEGER	No	No	None	None
dti_ratio	REAL	No	No	None	None
num_credit_lines	INTEGER	No	No	None	None
interest_rate	REAL	No	No	None	None
loan_term	INTEGER	No	No	None	None
loan_purpose	TEXT	No	No	None	None
status	TEXT	No	No	'REQUEST_RECEIVED'	None
created_at	TIMESTAMP	No	No	CURRENT_TIMESTAMP	None

`risk_assessments`

Append-only assessment history in normal API use, including input snapshot and model version. The latest applicable row supplies the current risk result.

Column	SQLite type	PK	Required	Default	Foreign key
id	INTEGER	Yes	No	None	None
loan_id	TEXT	No	Yes	None	loans.id (delete CASCADE)
model_version	TEXT	No	Yes	None	None
probability_of_default	REAL	No	Yes	None	None
risk_score	INTEGER	No	Yes	None	None
risk_level	TEXT	No	Yes	None	None
data_quality_score	INTEGER	No	Yes	None	None

Column	SQLite type	PK	Required	Default	Foreign key
anomaly_detected	INTEGER	No	Yes	0	None
review_level	TEXT	No	Yes	None	None
drivers_json	TEXT	No	No	None	None
protective_json	TEXT	No	No	None	None
dq_details_json	TEXT	No	No	None	None
anomaly_details_json	TEXT	No	No	None	None
assessed_at	TIMESTAMP	No	No	CURRENT_TIMESTAMP	None
input_snapshot_json	TEXT	No	No	None	None
reviewer_id	INTEGER	No	No	None	None

`funding_positions`

Saved simulated funding commitments. JSON holds the lender positions; the API prevents a second commitment for the same loan.

Column	SQLite type	PK	Required	Default	Foreign key
id	INTEGER	Yes	No	None	None
loan_id	TEXT	No	Yes	None	loans.id (delete CASCADE)
mode	TEXT	No	Yes	None	None
herfindahl_index	REAL	No	No	None	None
concentration_rating	TEXT	No	No	None	None
positions_json	TEXT	No	Yes	None	None
funded_at	TIMESTAMP	No	No	CURRENT_TIMESTAMP	None

`repayment_schedules`

Versioned repayment plans. The row ID is the version sent when recording payments. The latest row is active.

Column	SQLite type	PK	Required	Default	Foreign key
id	INTEGER	Yes	No	None	None
loan_id	TEXT	No	Yes	None	loans.id (delete CASCADE)
schedule_type	TEXT	No	Yes	None	None
allocation_policy	TEXT	No	Yes	None	None
principal	REAL	No	Yes	None	None
annual_rate	REAL	No	Yes	None	None
term_months	INTEGER	No	Yes	None	None

Column	SQLite type	PK	Required	Default	Foreign key
indicative_emi	REAL	No	Yes	None	None
total_interest	REAL	No	Yes	None	None
total_repayable	REAL	No	Yes	None	None
installments_json	TEXT	No	Yes	None	None
created_at	TIMESTAMP	No	No	CURRENT_TIMESTAMP	None

payments

Recorded receipt events and actual lender allocations. New events include schedule linkage and idempotency reference.

Column	SQLite type	PK	Required	Default	Foreign key
id	INTEGER	Yes	No	None	None
loan_id	TEXT	No	Yes	None	loans.id (delete CASCADE)
installment_num	INTEGER	No	Yes	None	None
amount_paid	REAL	No	Yes	None	None
paid_date	TIMESTAMP	No	No	CURRENT_TIMESTAMP	None
allocations_json	TEXT	No	Yes	None	None
schedule_id	INTEGER	No	No	None	None
idempotency_key	TEXT	No	No	None	None

audit_logs

Application events with entity, reviewer and JSON context. These records are not tamper-proof or cryptographically chained.

Column	SQLite type	PK	Required	Default	Foreign key
id	INTEGER	Yes	No	None	None
event_type	TEXT	No	Yes	None	None
entity_id	TEXT	No	No	None	None
reviewer	TEXT	No	No	None	None
details_json	TEXT	No	No	None	None
created_at	TIMESTAMP	No	No	CURRENT_TIMESTAMP	None

reviewer_accounts

Actual login accounts. Salt and password_hash must never be included in API responses or published backups.

Column	SQLite type	PK	Required	Default	Foreign key
id	INTEGER	Yes	No	None	None
username	TEXT	No	Yes	None	None
salt	TEXT	No	Yes	None	None
password_hash	TEXT	No	Yes	None	None
role	TEXT	No	Yes	None	None

reviewer_sessions

SHA-256 hashes of random session tokens, account link and expiry as Unix seconds. The raw token is not stored here.

Column	SQLite type	PK	Required	Default	Foreign key
token_hash	TEXT	Yes	No	None	None
account_id	INTEGER	No	Yes	None	reviewer_accounts.id (delete NO ACTION)
expires_at	REAL	No	Yes	None	None

review_decisions

Human decisions linked to a particular assessment and authenticated reviewer, with a required review note.

Column	SQLite type	PK	Required	Default	Foreign key
id	INTEGER	Yes	No	None	None
loan_id	TEXT	No	Yes	None	loans.id (delete NO ACTION)
assessment_id	INTEGER	No	Yes	None	risk_assessments.id (delete NO ACTION)
decision	TEXT	No	Yes	None	None
note	TEXT	No	Yes	None	None
reviewer_id	INTEGER	No	Yes	None	reviewer_accounts.id (delete NO ACTION)
created_at	TEXT	No	No	CURRENT_TIMESTAMP	None

schema_migrations

Additive migration marker. Version 2 identifies the integration schema; this is not a general migration framework.

Column	SQLite type	PK	Required	Default	Foreign key
version	INTEGER	Yes	No	None	None
applied_at	TEXT	No	No	CURRENT_TIMESTAMP	None

Indexes and transaction rules

Primary keys and username uniqueness create SQLite indexes. The integration adds a unique partial index named `payment_request_key` on non-NULL `payments.idempotency_key`. This allows historical NULL references while rejecting duplicate new keys. The schema does not declare a unique loan_id on funding or a foreign key on the added payment schedule column; workflow guards supply those checks.

`get_connection()` enables foreign keys and uses a transaction context, committing successful work and rolling back exceptions before closing. Profile edits, review decisions and receipt operations use explicit write transactions where needed. Payment recording performs duplicate checks, schedule checks, exact-cent allocation, insertion and installment-state update together.

JSON column contracts

Column	Stored content	Consumer
assessment drivers/protective JSON	Policy-rule factors with label, feature, value and fixed impact points	Analysis and report
`dq_details_json`	Completeness, missing inputs, warnings, contradictions and verification readiness	Queue, review guard and ORA
`anomaly_details_json`	Anomaly indicators and review context	Analysis and report
`input_snapshot_json`	Normalized borrower fields used for the saved assessment	Evidence and traceability
`positions_json`	Simulated lender IDs/names, invested amounts and shares	Funding, repayment and exposure
`installments_json`	Due dates, planned principal/interest, lender allocations and actual paid state	Repayment UI and receipt checks
`allocations_json`	Actual lender principal, interest and total allocation for one receipt	Outstanding balances and audit
`details_json`	Event-specific metadata	Audit inspection

JSON is serialized text, not separate relational child tables. Validation and aggregation happen in Python. Direct SQL edits can violate assumptions the API normally enforces. Do not edit financial JSON to make a dashboard total appear correct.

Migration and legacy records

Startup creates missing tables, checks columns before adding snapshot/reviewer and schedule/reference fields, and creates the partial reference index. It maps historical `FUNDING_OPTIMIZED` to `FUNDED` and `FULLY_PAID` to `CLOSED`. It does not delete old assessments, payments or schedules.

Legacy assessment snapshots and reviewer links can be NULL. They cannot retroactively prove who supplied the input. Historical payment rows without schedule IDs are ambiguous, particularly when the same installment number occurs more than once. The API reports `requires_reconciliation=true`, returns unknown outstanding principal and blocks new payments for those cases. A reviewed reconciliation process remains future work; no migration guesses allocations or removes duplicates automatically.

Accounting invariants

For each new receipt, amount paid equals the sum of lender total allocations. Each lender total equals principal plus interest. Cumulative receipts for an installment cannot exceed its scheduled payment. Outstanding principal equals original principal less all recorded principal allocations, while total received includes both principal and interest. Schedule changes are blocked after receipts to preserve these relationships.

Amounts are stored as SQLite REAL and monetary JSON numbers for compatibility. The payment service converts to integer cents for allocation. This is not a double-entry accounting schema and does not model bank settlement, refunds, write-offs or accrued late fees.

Backup and inspection

Stop the server before copying its database, or use SQLite's backup API for a live consistent snapshot. Keep backups outside Git with restricted access. Restore to a separate filename, set `RISKORA_DB_PATH`, and verify workflows before replacing the active file. Back up before upgrades; rollback is restoration of a reviewed application/artifact/database set, not an automatic down-migration.

Useful read-only checks on an offline copy include `PRAGMA integrity_check`, `PRAGMA foreign_key_check`, table row counts and reconciliation of each receipt's JSON allocations. Passing integrity checks proves storage consistency, not the correctness of historical business events. See README for the recovery procedure and TESTING.md for database isolation.

5. API contracts

Source: API_DOCUMENTATION.md

All business routes require the `riskora_session` cookie. Every POST must include `Content-Type: application/json` and `X-Riskora-Request: 1`. Browser calls must be same-origin. Unauthenticated requests return 401, forbidden actions 403, invalid inputs 400, missing records 404 and invalid workflow transitions 409. VIEWER accounts can read; REVIEWER and ADMIN accounts can perform case actions. Only ADMIN can create accounts.

Accounts

Route	Purpose
GET `/api/health`	Public startup/service status
GET `/api/auth/setup`	Public first-use setup status
POST `/api/auth/setup`	One-time loopback-only administrator creation: `username`, `password`
POST `/api/auth/login`	Credentials: `username`, `password`; sets HttpOnly SameSite session cookie
GET `/api/auth/me`	Current account ID, username and role
POST `/api/auth/logout`	Revoke current session
POST `/api/auth/accounts`	Admin creates `username`, `password`, `role` (ADMIN/REVIEWER/VIEWER)

Usernames have 3-80 characters; passwords have 12-200. Password hashing uses PBKDF2-HMAC-SHA256 with 600,000 iterations and random salts. Sessions last eight hours. Login is limited to ten attempts per source IP in five minutes in the local server process.

Borrower and risk workflow

Route	Contract
GET `/api/schema`	Numeric bounds, integer flags, categories and required intake fields
GET `/api/loans`	Queue, `search` and `filter=all/high/analyzed/incomplete`
POST `/api/loans`	`name` and canonical/aliased profile; returns `loan_id`, `borrower_id`
GET `/api/loans/{id}`	Saved profile, current assessment, funding, schedule, decision, ORA and ledger
POST `/api/loans/{id}/edit`	Full replacement profile plus `name`; only before funding; requires reassessment
POST `/api/risk/analyze`	`loanId` assesses saved fields; without ID, assesses an unsaved profile
POST `/api/risk/simulate`	`baseLoanId` and `scenario` overrides; no persistence
POST `/api/loans/{id}/review`	`assessment_id`, `decision`, `note`; latest assessment required
GET `/api/loans/{id}/audit`	Case events and authenticated review history

Review decisions: `APPROVED`, `REJECTED`, `NEEDS_VERIFICATION`. Notes must contain 5-2000 characters. Approval requires complete model inputs and passing input-quality rules. It records the human review; input-quality rules do not verify external documents.

Example complete loan body:

```
{
  "name": "Example borrower", "Age": 38, "Income": 720000,
  "LoanAmount": 40000, "CreditScore": 782, "DTIRatio": 0.16,
  "MonthsEmployed": 72, "NumCreditLines": 2, "InterestRate": 9.5,
  "LoanTerm": 12, "Education": "Master's", "EmploymentType": "Salaried",
  "MaritalStatus": "Married", "HasDependents": "Yes", "HasMortgage": "Yes",
  "HasCoSigner": "Yes", "LoanPurpose": "Education"
}
```

Fields required at intake: Income, LoanAmount, CreditScore, DTIRatio, LoanTerm, MonthsEmployed, NumCreditLines and InterestRate. Other omissions are stored as missing and prevent approval until resolved. LoanAmount must have at most two decimal places. The schema endpoint supplies authoritative ranges/categories.

Saved risk analysis cannot silently override stored profile values. Use the edit route for real corrections or simulation for hypothetical changes. ORA uses the same saved model risk band; its digest identifies the normalized snapshot and is not a verification certificate.

Funding, schedules and payments

POST `/api/funding/optimize`: `amount`, `mode=auto/single/fractional`, `lenderCount=2..6`. Omit `loanId` for a preview. Including `loanId` commits only if the loan is approved and amount equals the saved principal. The saved assessment supplies borrower risk. Existing commitments cannot be overwritten.

POST `/api/repayment/generate`: `loanId`, `principal`, `rate`, `term`, optional `startDate` (YYYY-MM-DD), `scheduleType=monthly/bi-monthly/flexible`, `allocationPolicy=pro-rata/largest-first/earliest-first`. Saved funding supplies lender positions. Principal, rate and term must match the approved loan. The response includes schedule `id`; another schedule is allowed only before any receipts.

POST `/api/repayment/record`:

```
{
  "loanId": "RETURNED_LOAN_ID", "scheduleId": 1, "installmentNum": 1,
  "amount": "1000.00", "idempotencyKey": "receipt-reference-unique-001"
}
```

Use the actual saved loan ID and current schedule ID. The amount must be positive, have at most two decimal places and not exceed the remaining installment amount. The payment reference must have 8-100 characters. Retries must use the same reference: matching submissions are a no-op, conflicting reuse returns 409. Partial receipts return PARTIAL; full installments return PAID. All installments paid moves the loan to CLOSED.

Loan details include `ledger.received`, `principal_paid`, `outstanding_principal`, `lender_principal_paid` and `payment_count`. Historical receipts without schedule linkage set `requires_reconciliation=true`, provide a warning and return null outstanding principal. Additional payments on such loans are blocked.

Analytics

- GET `/api/exposure/portfolio`: latest portfolio risk bands and recent funding structures.

- GET `/api/dataset/explorer`: labelled synthetic data with separate heuristic estimates; `search`, `risk`, `sort`, `page`, `limit`.

- GET `/api/model/metrics`: stored evaluation artifact.

The printable report is assembled in the browser from saved loan details; there is no remote certificate-generation or money-transfer API. See integration notes.

Request conventions and response handling

The API is unversioned and served from the same origin as the UI. JSON bodies must be objects. Send the session cookie returned by login on later calls; there is no bearer-token API. Do not store passwords in scripts committed to Git. Most successful operations return 200; loan creation and account setup/creation return 201. Errors contain `error` and `status` fields. Clients should show the server's reason and keep the user's unsaved input when correction is possible.

Status	Meaning	Client action
400	Invalid body, field, category, amount or query	Correct input; do not retry unchanged
401	No valid session	Sign in and reload saved state
403	Role, origin, request-header or setup restriction	Check access and request origin
404	Unknown record or unavailable static path	Check ID/path
409	Stage, assessment, schedule or receipt conflict	Reload the case and resolve the conflict
429	Login attempt limit	Wait for the five-minute window to pass
500	Unexpected server failure	Inspect local logs; do not assume a write failed before retrying

A network timeout does not prove a POST was not committed. Loan creation has no client idempotency token, so inspect the queue before repeating it. Receipt recording does have an idempotency reference; retry with the same reference and original values.

Detailed read semantics

The queue response contains `count` and `loans`. `search` matches loan ID, borrower name, amount and purpose. `high` uses the latest visible HIGH assessment, `analyzed` requires a visible assessment, and `incomplete` evaluates current input readiness. Latest-assessment selection prevents one queue row per historical assessment. A profile reset to REQUEST_RECEIVED does not show a stale risk result.

Loan detail returns the saved borrower and loan fields together with decoded assessment, funding, repayment, review decision, ORA and ledger objects. Missing stages can be null. The frontend must clear a previous case's panels when those objects are absent. Old review history remains history; it does not authorize a new assessment automatically.

The audit endpoint returns up to 100 case events and review decisions joined to reviewer usernames. Generic successful API-action audit entries may use an endpoint path as entity ID and therefore are not all included in a loan-specific audit query. Audit records are application evidence, not an immutable external log.

Portfolio exposure aggregates current stored loan/assessment/funding information. Dataset explorer pagination and filtering concern synthetic CSV rows, not saved applications. The metrics endpoint reads saved evaluation metadata; it does not perform fresh training or evaluation.

Simulation contract

```
{"baseLoanId":"RETURNED_LOAN_ID","scenario":{"LoanAmount":35000,"DTIRatio":0.14}}
```

Only supplied canonical or aliased scenario fields replace the saved baseline. An empty scenario must reproduce the baseline and return zero difference. Simulation does not edit the borrower, save approval, commit funding or change repayment terms. Use the profile-edit endpoint for actual corrections before funding.

Financial edge cases

`bi-monthly` means twice monthly in this application, with dates on the 5th and 20th; it does not mean one payment every two months. `flexible` adds a six-day due window. These are demonstration scheduling policies, not jurisdiction-specific loan contracts.

New payment references are globally unique across stored receipts. A matching duplicate returns success with `duplicate=true` and the existing payment ID. A different loan, installment or amount under the same reference is rejected. A stale schedule ID is rejected for a new receipt; clients should restore the saved plan after refresh.

Partial payment allocations are proportional to the remaining scheduled lender principal/interest buckets, with integer-cent remainders distributed deterministically. This is not an interest-first collections policy. Overpayments are rejected rather than automatically applied to a later installment. Refunds, reversals, early payoff, fees and external settlement callbacks are not implemented.

See DATA_DICTIONARY.md for units and all field ranges, and DATABASE_SCHEMA.md for persistence and legacy limitations.

6. Machine learning pipeline

Source: ML_PIPELINE.md

The pipeline is an explicit offline training process plus a read-only inference service. Starting RISKORA loads saved artifacts; it does not regenerate data or train automatically. The current dataset is synthetic and does not establish real-world lending performance.

Data origin and target

`ml/generate_data.py` generates 12,000 rows using a fixed seed of 42. The 18 CSV columns consist of `LoanID`, 16 borrower/loan predictors and `Default`. Numeric values and categories are sampled from programmed distributions; the default label is also generated by programmed risk assumptions and randomness. There is no observed bank repayment history or independently verified customer sample in this dataset.

`Default=1` means the synthetic positive class; `Default=0` means the synthetic negative class. The target does not define a validated real-world default horizon such as 90 days past due within a year. Any deployment must first define and collect the intended outcome.

Reproduction commands

Use the pinned Python environment from README. Run these only when intentionally creating a new data/model revision:

```
python -m ml.generate_data
```

```
python -m ml.train
```

```
python run.py --check
```

```
python -m unittest discover -s tests -v
```

Generation and training overwrite project data/artifacts. Preserve the existing revision first, perform experiments in a separate checkout and review resulting metrics before accepting changes. Seeded code makes the procedure repeatable under a matching environment; bit-for-bit output across different numerical libraries or platforms is not guaranteed.

Split and feature preparation

The training script uses stratified train/validation/test splits with random state 42: 8,400 training rows, 1,800 validation rows and 1,800 test rows. The test set is held aside from candidate fitting. The data is synthetic and independently sampled; this split is not a substitute for temporal or external validation on real lending cohorts.

The 16 original predictors include nine numeric inputs and seven categorical inputs. Two derived numeric inputs are added:

```
LoanToIncome = round(LoanAmount / max(Income, 1), 4)
```

```
MonthlyBurdenRatio = round((LoanAmount / LoanTerm) / max(Income / 12, 1), 4)
```

Income is annual, and term is months. MonthlyBurdenRatio is a principal-only burden proxy; it is not an amortizing EMI-to-income ratio and does not include interest.

Eleven numeric features are standardized. Seven categorical features are one-hot encoded with the first category dropped and unknown categories ignored by the transformer. The resulting artifact expects 26 transformed columns. API category validation is stricter than the transformer's unknown-category setting.

The preprocessor is fitted on the outer training partition. It is then applied before the calibration cross-validation step. This means preprocessing is not refitted independently within every calibration fold; a future training revision should place the full preprocessing pipeline inside cross-validation.

Candidate models and selection

Candidate	Main configuration	Validation ROC AUC	Validation average precision	Validation Brier
Logistic regression	C=0.5, balanced classes, max_iter=1000	0.8133	0.4546	0.1687
Random forest	120 trees, depth 10, leaf minimum 5, balanced classes	0.8164	0.4343	0.1380
HistGradientBoosting	learning rate 0.08, 150 iterations, depth 6, leaf minimum 15	0.7994	0.4263	0.0816

The script explicitly selects HistGradientBoosting and applies sigmoid calibration through five-fold `CalibratedClassifierCV`. It does not automatically choose the highest-AUC candidate. HistGradientBoosting has the lowest recorded validation Brier score among these candidates, but claiming it wins every metric would be incorrect.

`pr_auc` in the metrics artifact is calculated as average precision. Keep that distinction when explaining the value; it is not necessarily trapezoidal area under a plotted precision-recall curve.

Final held-out results

The calibrated saved model has test ROC AUC 0.7919, average precision 0.4023 and Brier score 0.0821. At probability threshold 0.20, precision is 0.362, recall 0.402 and F1 0.381. The confusion matrix contains TN=1460, FP=141, FN=119 and TP=80.

The threshold represents a configured policy with a 4:1 cost assumption in metadata, not a proven economically optimal decision boundary. The low/high band thresholds are 0.08 and 0.20. Human approval is separate from these classifications and is never inferred solely from a low band.

Artifacts and startup validation

File	Function
`ml/artifacts/riskora_model.joblib`	Fitted calibrated binary classifier
`ml/artifacts/preprocessor.joblib`	Fitted scaling and encoding transformer
`ml/artifacts/model_metrics.json`	Version, splits, evaluation, thresholds and reference summaries

Startup requires all three nonempty files. It treats scikit-learn version mismatch warnings as errors, checks metadata thresholds and version, transforms a probe input, checks the transformed feature count and verifies a finite, normalized two-class probability output. The expected class order is `[0, 1]`.

These checks detect important compatibility failures but are not a cryptographic artifact-signing system. Only load artifacts from a trusted reviewed revision; joblib uses Python serialization and can execute code during loading.

Inference and explanation sources

`backend/schema.py` normalizes API, database and scenario aliases. Missing values remain missing in stored profiles and quality reports. `inference_profile` applies documented fallbacks only when producing an inference input. A fallback-assisted assessment must not be presented as a fully verified borrower record.

`ml/infer.py` adds derived features, transforms the row, calls `predict_proba`, assigns a band and produces a display score. With PD p, low threshold L and high threshold H:

```
p < L: score = round(30 * p / L)

L <= p < H: score = round(30 + 30 * (p - L) / (H - L))
```

```
p >= H: score = round(60 + 39 * min(1, (p - H) / (0.60 - H)))

final score = max(5, min(99, score))
```

The fixed rule points shown next to credit, DTI, employment and other factors are separate policy indicators. Their sum does not reconstruct the model score or probability. The saved coefficient list is from the logistic-regression baseline and does not explain the calibrated HistGradientBoosting model. ORA summarizes these sources without replacing the model band.

The dataset explorer uses its own labelled heuristic estimates. It should not be used to demonstrate exact model parity. For parity, compare a complete saved profile, the same direct analysis input and an unchanged scenario under the same artifact revision.

Work before real-data use

Define the outcome and observation window, obtain lawful representative data, separate applicants chronologically, measure missingness and selection bias, compare against simple baselines and evaluate calibration by relevant subgroups. Refit preprocessing inside cross-validation, select thresholds against measured costs, add model-specific explanations and establish versioned approval and rollback procedures. Drift monitoring requires observed future inputs and outcomes; it is not currently operating in this prototype.

See MODEL_CARD.md for the intended use and evaluation limits, and DATA_DICTIONARY.md for every input and fallback.

7. Model evaluation and limitations

Source: MODEL_CARD.md

Identity and intended use

Item	Current artifact
Model version	`riskora-ml-v1.0`
Algorithm	HistGradientBoosting with sigmoid CalibratedClassifierCV
Recorded training time	2026-09-16T05:28:28Z
Dataset	12,000 synthetic loan rows generated by project code
Split	8,400 train / 1,800 validation / 1,800 test
Inputs	16 raw predictors, two engineered ratios, 26 transformed columns
Output	Probability of synthetic Default=1, risk band and derived display score
Intended use	Academic demonstration and reviewer decision-support workflow
Excluded claims	Validated real-world underwriting, bureau scoring or autonomous lending

The model is packaged in `ml/artifacts/riskora_model.joblib`, with `preprocessor.joblib` and `model_metrics.json`. Keep the three files together. The recorded version identifies this artifact family; future retraining should also record data, code and artifact hashes rather than reusing the same version string without a release record.

Data and representativeness

The generator samples borrower characteristics and creates labels from programmed relationships. Names in the demonstration queue and rows in the training CSV are not evidence of a real lending cohort. The dataset has no verified repayment timeline, real default observation window or independently measured collection process.

Synthetic data is useful for testing schema handling, model plumbing and the review workflow. Performance against labels created by the same generator does not establish accuracy on real applicants. A random split also does not test future economic periods, geographic transfer or deployment selection effects.

Evaluation results

Metrics below are copied from the saved artifact, not a new training run during documentation generation.

Test metric	Value	Interpretation
ROC AUC	0.7919	Ranking across thresholds on this synthetic test set
Average precision (`pr_auc`)	0.4023	Precision-recall summary on this test distribution
Brier score	0.0821	Mean squared error of predicted probabilities
Precision at 0.20	0.362	Share of flagged cases with synthetic default labels
Recall at 0.20	0.402	Share of synthetic defaults flagged
F1 at 0.20	0.381	Harmonic mean of precision and recall

Actual / predicted	Non-default	Default
Actual non-default	1,460	141
Actual default	119	80

There are 199 defaults among 1,800 test rows. Accuracy is 85.56%, while predicting every case as non-default gives 88.94% accuracy. This illustrates why accuracy alone is misleading here. The selected threshold misses 119 of 199 synthetic defaults, about 59.8%; a high accuracy headline would hide that limitation.

Risk band	Probability interval	Test cases	Defaults	Observed synthetic default rate
LOW	PD < 0.08	960	36	3.75%
MEDIUM	0.08 <= PD < 0.20	619	83	13.41%
HIGH	PD >= 0.20	221	80	36.20%

Band-level observed rates are not guarantees for an individual borrower. No confidence intervals, external validation or subgroup calibration study are supplied with this artifact. Brier score alone does not establish calibration for every subgroup.

Thresholds and human decisions

The configured thresholds are 8% and 20%. The metadata describes a 4:1 cost assumption; the project does not show a validated operational cost study or proof that 20% maximizes lending utility. Classification and approval are separate. Even a LOW case must pass input-quality checks and receive a recorded human decision before funding can be committed.

The display score is a piecewise transformation bounded to 5-99. It is not a CIBIL or other bureau score. Fractional funding changes lender concentration but must not change the borrower's predicted probability or band.

Explanation and evidence limits

Policy indicators compare selected inputs with fixed thresholds and assign fixed points. Those points are not SHAP values, causal effects or a decomposition of the trained prediction. Their sum need not equal the display score. The global coefficient chart belongs to the logistic-regression baseline, not to the calibrated tree model.

ORA uses saved inputs and assessment results to make a deterministic evidence brief. Its SHA-256 digest identifies a normalized input snapshot; it does not certify that the supplied values are true. There are no bank, identity, psychometric or eNACH verification integrations. Dataset-explorer estimates are separate heuristics and must not be quoted as trained-model predictions.

Missingness and safeguards

Creation requires eight fields; approval requires all 16 and acceptable internal consistency. Missing optional intake fields are retained as missing. Inference uses documented fallback values so an incomplete case can still produce a provisional assessment, but data-quality flags route it for verification. Users must not treat fallback values as supplied facts.

Startup validates the matching artifact set, feature count, class order and finite probability output. API validation rejects invalid ranges and categories. Saved assessments include an input snapshot, model version and reviewer reference. Human decisions refer to the latest assessment ID. These safeguards improve traceability; they do not validate real-world model fitness.

Known limitations and release conditions

The model has no demonstrated fairness analysis, real-data calibration, independent test cohort, drift monitoring or production performance benchmark. Preprocessing is fitted on the outer training set before calibration folds; moving preprocessing inside cross-validation would improve fold isolation. The choice of final model is explicit in code rather than an automated selection procedure.

Before use for real lending, define the outcome and horizon, validate lawful data collection, evaluate representative historical outcomes, test temporal and subgroup behavior, compare against simple baselines, choose thresholds against actual costs and establish model review/rollback ownership. Keep the system in decision-support mode with human accountability. See ML_PIPELINE.md for reproduction and PROJECT_ANALYSIS.md for implementation priorities.

8. OraAgent integration

Source: INTEGRATION.md

Source and integration choice

The supplied `RiskOra_OraAgent-main.zip` contains a separate FastAPI application under `riskora/`, a browser frontend, another model pipeline, CSV files and review documents. RISKORA already has a working Tornado application, a shared input schema and regression coverage. Running both backends would create two incompatible loan stores and prediction contracts.

The useful workflows were adapted into the existing application. The ZIP was inspected in `tmp/incoming`; its entry scripts and serialized models were not executed. The incoming archive remains unchanged. Its SHA-256 is `ee76c66477ad06efa4fcef5b3042b17abc0e722a843f2593e2adb2f2266f29cd`.

Incoming component	Integrated result
`riskora/backend/security.py`	Adapted salted password hashing and login into `backend/security.py`; replaced hardcoded signing secrets/custom JWTs with revocable random sessions.
Borrower application routes and frontend form	Added create/edit dialogs driven by the existing shared schema; persistence uses the current borrower/loan tables.
`services/risk_service.py` ORA analysis	Adapted as `backend/services/ora_service.py`, based on the saved calibrated assessment and actual snapshot. Removed invented identity, bank, device, psychometric and mandate evidence.
Reviewer/marketplace workflow	Added explicit review records linked to the current assessment and authenticated reviewer before funding.
Funding and repayment services	Kept the established tested allocation engine; integrated strict stage guards, approved terms, schedule IDs and exact-cent partial receipts.
Report/certificate features	Extended the existing printable report with ORA, reviewer decisions, receipts and outstanding principal. No compliance certificate is asserted.

The incoming FastAPI runner, incompatible integer-ID database schema, alternate model binaries, mock lender marketplace, negotiation routes, fabricated TLS/domain responses and legal compliance assertions were not copied into the runtime. Mock bank/eNACH/psychometric values would otherwise appear to be verified facts. The existing model's risk bands remain authoritative; ORA does not apply a contradictory 300-900 bureau-style score.

New modules

- `backend/security.py`: salted PBKDF2 passwords and hashed revocable sessions.

- `backend/integration.py`: first-use setup, account roles, shared form schema, profile edits, decisions and review history.

- `backend/services/ora_service.py`: evidence brief and input digest.

- `backend/services/payment_service.py`: integer-cent proportional partial receipts against remaining scheduled principal/interest allocations.

- `integration.js`: login, borrower intake, review, account and payment dialogs.

- `tests/test_ora_integration.py`: security, lifecycle, snapshot, payment and migration regressions.

The original source of each adapted feature is identified above. The integration does not introduce an LLM, a bank connection, autonomous credit approval or an external payment processor.

Data compatibility

Database changes are additive. Existing borrowers, loans, schedules and receipts are retained. Separate account/session tables avoid treating historical reviewer labels as authenticated credentials. Assessment snapshots and reviewer IDs are nullable for historical records; new assessments populate both.

New receipt records carry a schedule ID and unique idempotency reference. Older receipts cannot be safely assigned to a schedule merely from an installment number, especially where that number appears repeatedly. Those loans display a reconciliation warning and block new payments. Their original receipts remain available in SQLite for review. No historical receipt was deleted or guessed away.

The default mutable database now lives under ignored `data/runtime/`. On first default startup it is copied from the former database. Tests and browser checks use separate temporary databases; the original application database remains unchanged by the integration checks.

Behaviour contracts

- Editing an unfunded profile invalidates its displayed assessment and decision eligibility. History is retained.

- Approval requires the latest assessment ID, complete valid model inputs and a nonempty review note.

- Funding requires approved status and the saved amount. The saved model supplies the risk band.

- Schedule generation uses saved lender positions and approved loan terms. It cannot replace a schedule after any payment.

- Partial receipts split integer cents across remaining scheduled lender principal/interest amounts. Every receipt and final installment reconciles exactly.

- A duplicate reference with the same loan, installment and amount is a no-op; a conflicting use is rejected.

- Exposure comes from stored receipt allocations. Model risk remains separate from lender concentration.

- Reports use the current risk band, not approval-label string matching. This corrects an old low-risk report that incorrectly showed a high-risk warning.

See README for startup and TESTING.md for verification results. Production deployment and real-data model validation remain separate work.

9. Demonstration procedure

Source: DEMO_GUIDE.md

Use this walkthrough for a faculty review of the integrated application. It demonstrates a complete recorded workflow using synthetic inputs. Do not describe funding or payment buttons as real bank transactions.

Prepare a clean demonstration

Install and run the checks in README. Use a separate runtime database so repeated demonstrations do not change the normal reviewer database:

```
$env:RISKORA_DB_PATH = 'data/runtime/review-demo.db'
```

```
.\.venv\Scripts\python.exe run.py 8010
```

A custom path avoids importing old historical receipts. The server creates the schema and six example loans when empty. Open the printed address on the same computer, complete the one-time administrator setup and sign in. Do not display or include the password in screenshots. Use an existing demonstration account if this database was already initialized.

Before presenting, check `python run.py --check`, browser connectivity and the loan queue. Keep the model card available so metrics can be explained from their source. The bundled historical review PDF predates integration; use the current root review document for current behavior.

Main demonstration: create through partial repayment

1. Login and describe the roles

Show the login screen. Explain that accounts are persisted, sessions expire, and a viewer cannot modify loans. An administrator can create a reviewer or viewer account. Authentication is enforced by the API as well as the interface.

2. Create a complete borrower

Use the create-loan action and enter this synthetic profile:

Field	Value
Borrower name	Faculty demo borrower
Age	38
Annual income	INR 720,000
Loan amount	INR 40,000
Credit score	782
Employment months	72
Credit lines	2
Annual interest	9.5%
Loan term	12 months
DTI	0.16
Education	Master's
Employment	Salaried
Marital status	Married
Dependents, mortgage, co-signer	Yes for each
Purpose	Education

Save and note the generated loan ID. Reopen it from the queue to show persistence. Explain that 0.16 means 16% DTI and that income is annual, not monthly. The backend uses one schema regardless of entry point.

3. Assess the saved profile

Run risk assessment. With the current artifact, the previously checked example produced approximately 0.52% PD, a display score of 5 and LOW risk. These values identify this artifact/profile combination; do not promise them after retraining.

Show input-quality results alongside risk. Explain that quality is about completeness and consistency, whereas PD is a model output. Point out that driver/protective points are fixed policy indicators. The synthetic training data does not validate actual lending decisions.

4. Explain ORA and record human review

Open the decision/review section. ORA summarizes saved input evidence and the model result. Its snapshot digest helps identify an input version; it does not prove income or identity. Record approval with a meaningful note such as: "Complete synthetic demonstration inputs reviewed; proceeding with the workflow exercise."

Show the recorded reviewer and decision. Explain that the review is linked to the latest assessment. A model recommendation alone cannot commit funding.

5. Preview and commit funding

Select fractional funding with three lenders. Discuss concentration before saving. The current sample structure uses about 43.48%, 33.33% and 23.19% participation. Commit the funding explicitly. Confirm that the borrower PD and band remain unchanged.

The lenders are generated demonstration positions. This does not connect to a lender marketplace or disburse the requested amount.

6. Generate a repayment schedule

Choose monthly repayment and pro-rata allocation with a suitable start date. The principal, rate and term must remain the approved INR 40,000, 9.5% and 12 months. Explain that the schedule is a plan and has a saved version ID.

The checked example's first installment was INR 3,507.34. Rounding may adjust a final installment to reconcile the plan. Show each installment's principal, interest and lender allocation rather than only the total EMI.

7. Record a partial receipt

Record INR 1,000 against the first installment using a new reference such as `faculty-demo-20261010-001`. Do not reuse that reference for another payment. The checked example allocated INR 909.71 to principal and INR 90.29 to interest, leaving INR 39,090.29 outstanding principal.

Explain why the outstanding balance did not fall by the full INR 1,000: interest is received cash but does not repay principal. The installment should show PARTIAL and the loan should show REPAYING. The remaining installment amount is INR 2,507.34.

8. Reload and report

Refresh the page and reopen the same case. Confirm the saved funding, schedule, receipt and outstanding principal remain present. Navigate to exposure and the report. Print the case report or save it as PDF through the browser. The report should describe the LOW band appropriately and include current saved financial details.

End by signing out. The authenticated shell should no longer be available. This completes the path from account access to a persisted repayment event and report.

Short negative demonstrations

Demonstration	Expected result	What it proves
Omit an optional input, save and assess	Needs verification; approval blocked	Missing data is surfaced
Supply a negative loan amount	Validation error	Backend enforces numeric rules
Analyze an unchanged scenario	Zero delta	Saved baseline and mapping agree
Try to fund before approval	Conflict response	Human review gate
Try to edit a funded profile	Conflict response	Approved financial terms protected
Retry identical payment reference	No second receipt	Idempotent recording
Reuse reference with another amount	Conflict response	Reference cannot disguise a different payment
Try to pay more than remaining installment	Validation error	Overpayment blocked
Try to regenerate schedule after payment	Conflict response	Receipt history remains linked
Access source/database through static URL	Not served	Restricted static routing

Use a fresh demo case for destructive-looking attempts so the main presentation case remains clear. Automated tests are the more reliable place to demonstrate all invalid combinations.

If the demonstration is interrupted

Read the actual server URL if port 8010 was occupied. A login timeout requires signing in again, not recreating the database. If a request times out, reopen the case before repeating a write. Retry receipts with the same reference. For historical demo cases with reconciliation warnings, explain the limitation and use a newly created case; do not delete their old receipts to remove the warning.

After stopping the demo server, remove the temporary environment setting with `Remove-Item Env:RISKORA_DB_PATH`. Keep or securely archive the demo database locally. Runtime databases and credentials must not be committed.

10. Verification and evidence

Source: TESTING.md

The supported automated runner is Python's built-in unittest. Tests use temporary SQLite databases; never run manual API demonstrations against a database whose records must remain unchanged. Use the pinned Python 3.14 environment from README.

Commands

```
.\.venv\Scripts\python.exe -m pip check

.\.venv\Scripts\python.exe run.py --check

.\.venv\Scripts\python.exe -m unittest discover -s tests -v

node --check app.js

node --check integration.js
```

On Linux/macOS, replace the interpreter path with `.venv/bin/python`. To run one area, use `python -m unittest tests.test_ora_integration -v`. Node syntax checks do not execute browser logic or prove accessibility.

`pytest.ini` restricts optional pytest discovery to `tests/` and excludes extracted archives and generated outputs. pytest is not a runtime dependency and is not used by CI; install it separately if choosing that runner. The unittest command above remains the baseline reproduction command.

Database isolation

`tests/support.py` supplies `IsolatedDatabaseMixin`. Each database-backed test creates a TemporaryDirectory, patches `backend.database.DB_PATH`, initializes a new schema, seeds demo cases and creates its own authenticated test account/session. Cleanup restores the patch and removes the temporary directory. Test passwords are fixtures, not application credentials.

Startup tests use separate subprocess environment paths. Importing modules and running `run.py --check` must not create a database. The default-migration test builds its own temporary historical/runtime paths and checks preservation. No test requires a user to delete an existing runtime file.

To verify isolation independently, hash the application database before and after running the suite while the normal server is stopped or idle. Do not compare a database concurrently changing from real user actions and assume every difference came from tests.

Coverage map

Test module	Main checks
`test_foundation.py`	Static allowlist, full profile creation/reload, invalid-create rollback, JSON errors, normalization parity, unchanged scenarios, aliases, queue deduplication, incomplete filtering, DB lifecycle, stage preservation and artifact/startup failures
`test_api_integration.py`	Health, queue/detail, analysis, scenario, funding, repayment, explorer and payment HTTP contracts
`test_riskora.py`	Model bands, data quality, anomaly rules, funding concentration, amortization, twice-monthly dates and allocation reconciliation
`test_workflow.py`	Saved contract restoration, payment guards and exact allocations for each schedule policy
`test_ora_integration.py`	Login/logout, CSRF/origin controls, roles, expiry, one-time setup, approval snapshots, ORA evidence, edit invalidation, partial/duplicate receipts, approved terms, closure and migration preservation

The suite contains 63 tests at this documentation revision. The documentation release run on 10 October 2026 completed with all 63 passing in 29.148 seconds. This is a count of tests, not a claim of 100% code or behavioral coverage. Dependency consistency, artifact startup validation and both JavaScript syntax checks also passed locally.

Financial assertions

Tests check that a payment is positive and within the remaining installment, that sub-cent and nonfinite values fail, and that stale schedules cannot accept new receipts. Matching idempotency retries do not add a second payment; conflicting reference reuse fails.

For every accepted receipt, the allocated lender totals equal the receipt to the cent. Principal and interest reconcile within each allocation. Partial payments accumulate correctly, final payments close the loan and outstanding principal reaches zero. Schedules cannot be replaced after receipts. Legacy unlinked receipts remain in storage and explicitly require reconciliation.

These tests do not cover real bank settlement, refunds, reversals or write-offs because those features are not implemented. They also do not prove all possible concurrent interleavings; broader concurrency testing remains planned work.

Manual browser verification

The integration walkthrough used a separate temporary database and checked:

- First administrator setup, login and logout.

- Creation of a complete borrower and assessment from saved fields.

- Human approval tied to the assessment, followed by explicit fractional funding.

- Monthly schedule generation and a partial receipt.

- Saved funding, schedule and outstanding principal after page reload.

- ORA source labels and low-risk report wording.

- A 390 by 844 mobile viewport with navigation and no horizontal overflow.

- Browser console output during the exercised path.

The example in DEMO_GUIDE.md produced a INR 1,000 receipt split into INR 909.71 principal and INR 90.29 interest, with INR 39,090.29 outstanding. A screenshot is retained at `output/integration/RISKORA_ORA_integration.png`. It is evidence of one exercised state, not proof that every screen or accessibility path is correct.

Browser checks are manual and are not part of the current GitHub Actions workflow. Add automated browser tests for slow responses, case-switch races, session expiry, keyboard focus, validation errors and print output before claiming full end-to-end regression coverage.

CI

`.github/workflows/tests.yml` runs on pushes and pull requests with a Windows/Linux matrix and Python 3.14. It installs pinned runtime dependencies, runs `pip check`, validates startup artifacts, executes unittest and checks both JavaScript files using Node 22.

A local passing run does not prove that a new remote Actions run passed. Inspect the GitHub run after pushing. Dependency-install availability, operating-system differences and remote runner failures can still affect CI.

Documentation and PDF verification

Install optional `requirements-docs.txt` and run `python generate_final_review_pdf.py`. The generator reads selected maintained guide sections and model metadata without touching SQLite. Verify Markdown relative links, output existence and extracted PDF text. Render every page to check margins, table wrapping, diagram labels, headers and page breaks; text extraction alone cannot detect overlap or clipping.

The generator supports a fixed report date so repeated generation can use the same visible date. The generated document is a snapshot: regenerate after changes to source documentation or metrics. The older Review 2 PDF remains labelled historical.

Release checklist

1. Inspect the Git diff for credentials, runtime files and unintended incoming ZIP content.
2. Run dependency, startup, regression and syntax checks.
3. Exercise the affected browser flow if UI or API behavior changed.
4. Update API, schema and model claims against implementation.
5. Regenerate and visually inspect the current review PDF.
6. Verify that runtime databases and temporary outputs remain ignored.
7. Commit source and intended deliverables, push normally and check remote status.

Tests are a boundary for verified claims, not a replacement for review. Record failures and untested areas explicitly instead of describing the whole application as production-ready.

11. Remaining work and priorities

Source: PROJECT_ANALYSIS.md

This assessment describes the integrated repository, not every feature in the received ZIP. The project has a usable local reviewer workflow and a synthetic-data model demonstration. It is not yet a validated production lending service.

Current system

The frontend is plain HTML, CSS and JavaScript, served by the same Tornado process as the API. SQLite persists borrowers, loans, assessments, decisions, funding, schedules, receipts and accounts. The shared borrower schema keeps create/edit, stored analysis and simulation aligned. Trusted joblib artifacts provide inference without retraining at startup.

OraAgent features were adapted into this system rather than copied as a second backend. Login, intake forms, review records and an ORA brief are integrated with existing IDs and financial services. The integration avoids fabricated bank/identity evidence and keeps the saved model band authoritative.

Repository responsibilities and maintenance risks

Area	Current responsibility	Maintenance consideration
`app.js`, `index.html`, `styles.css`	Main workstation and reports	Large view functions make state changes harder to isolate
`integration.js`	Auth and new workflow dialogs	Keep event/state ownership explicit between both scripts
`backend/server.py`	Main HTTP handlers and orchestration	Split only after preserving API contracts with tests
`backend/integration.py`	Account, edit, review and audit routes	Route factory depends on shared BaseHandler
`backend/schema.py`	Single borrower normalization contract	Update tests, forms and dictionary together
`backend/services/`	Quality, anomaly, funding, schedule and receipt logic	Keep financial invariants independent of presentation
`backend/database.py`	Schema, migration and seed	Additive migration needs a managed upgrade plan
`ml/`	Synthetic generation, training and inference	Preserve artifact/data/code provenance per release
`data/`	Synthetic CSV and historical demo database	Live accounts/receipts must stay in ignored runtime storage
`tests/`	Service and HTTP regressions	Browser automation and concurrency coverage remain limited
`output/`	Review PDFs and integration evidence	Label historical reports to avoid stale claims
`tmp/`	Ignored extracts, logs and working files	Never make runtime imports depend on it

Dependency versions are pinned, Python 3.14 is specified, startup validates all three model artifacts and CI is configured for Windows/Linux. `.env.example` documents actual variables without introducing unused secrets or settings. A separate optional documentation dependency file keeps PDF generation out of the server installation.

Confirmed improvements in the integration

Loan creation validates and persists one normalized profile. The queue selects one latest assessment, and verification filtering uses current completeness and contradictions. Saved analysis and unchanged scenarios share a baseline. Static serving is restricted. Reviewer identity and assessment snapshots are recorded. Explicit human approval gates funding, and schedule generation uses approved terms.

New payments reject malformed, negative, duplicate-conflicting and excessive amounts. Partial receipts reconcile in cents and update principal exposure from actual allocations. Schedule versions are protected after payment. Reload restores saved finance details, and reports use the actual model band. Ambiguous historical receipts remain preserved and visibly require reconciliation.

These improvements are covered by the tests named in TESTING.md. They do not imply that every concurrency, accessibility or operational scenario is tested.

Remaining limitations

Limitation	Consequence
Synthetic training and test data	No real-world underwriting accuracy claim
Fixed policy explanations	No model-specific attribution or causal explanation
Deterministic ORA brief	No independent document verification or agent reasoning
Legacy unlinked receipts	Certain historical balances remain unknown until reconciliation
REAL amounts and JSON finance structures	API invariants are essential; not a full accounting ledger
No reversals/refunds/write-offs	Corrections to actual financial events lack a supported flow
Single process and synchronous work	Throughput and latency need measurement
Process-local login limiter	Not shared across workers or restarts
No password-reset/account-disable workflow	Account lifecycle requires further design
Additive startup migration	No managed rollback or historical schema test matrix
Public health is descriptive	Does not prove database readiness or backup health
No supported HTTPS deployment package	Local success is not deployment readiness
Browser checks are manual	UI regressions are not all caught by CI
Historical tracked demo database	Keep it a fixture; never replace it with a live database

Work in priority order

1. Correctness and accounting

Build a reviewed legacy-receipt reconciliation tool that preserves original events and records every correction. Define reversal, refund and early-payoff behavior before extending payment handling. Add adversarial concurrent-request tests around edit/assess/review/fund/schedule operations. Measure and close race windows rather than claiming all multi-user writes are fully serialized.

Completion means each receipt and correction has a traceable relationship to a schedule and lender allocation, and concurrent submissions cannot bypass stage or balance rules. Continue using exact-cent invariants and immutable event records where possible.

2. Complete the reviewer workflow

Decide whether separate persisted Draft and Submitted stages are necessary, then define the allowed transition table before changing UI labels. Add account recovery, account disable/session revocation controls and a supported reconciliation view. Add automated browser journeys for login, incomplete intake, approval, payment, refresh and print report.

Completion means a reviewer can finish or explicitly resolve every supported case state without direct database edits, and reload or session expiry does not lose committed work or leave stale displays.

3. Trustworthy ML

Define a real outcome and observation window, validate an appropriate dataset, split temporally and compare to simple baselines. Put preprocessing inside calibration folds. Evaluate subgroup behavior and probability calibration, choose thresholds against measured costs and add explanations appropriate to the actual trained model. Version code, data and artifact hashes together.

Completion means every claimed metric has a documented cohort, period, method and limitation, and a reviewer can identify the source of each score and explanation. Synthetic model results remain labelled until replaced by validated evidence.

4. Security and accountability

Review threat boundaries for remote use, configure TLS and trusted proxy behavior, add account lifecycle management and improve audit completeness. Review dependency updates against model serialization compatibility. Protect database files and backups, and document retention and access ownership. Add abuse, session and origin tests for the intended deployment topology.

Completion means unauthorized requests cannot retrieve private data or change workflow state, account changes are traceable, and backup/restoration does not silently revive unintended access. No compliance certification should be inferred from these engineering controls.

5. Deployment and maintenance

Provide a supported process-manager or container configuration, readiness checks, structured operational logs, managed schema migrations and a tested backup/restore runbook. Benchmark expected workload and choose a database/worker design based on measurements. Add automated release checks for documentation links and PDF regeneration.

Completion means a fresh host can be installed, upgraded, monitored and restored using documented commands, with recorded verification of the restored application and model revision.

Future features after the foundation

Lender integrations, document extraction, drift monitoring and agent-assisted review should use the same validation and permission boundaries. Any future agent must submit proposals or evidence through supported APIs and retain explicit human approval for lending and financial actions. Adding a chatbot before accounting and workflow gaps are resolved would not address the highest-priority needs.

Evidence and references

Implementation sources are `backend/schema.py`, `backend/server.py`, `backend/integration.py`, `backend/database.py`, `backend/security.py`, the services directory, `ml/train.py` and `ml/infer.py`. Evaluation values come from `ml/artifacts/model_metrics.json`. Tests and browser evidence are described in TESTING.md; integration provenance is in INTEGRATION.md. No external regulatory approval or production certification is asserted.